

PostgreSQL

Tek Veritabanı, Sonsuz Yetenek

Mimari, Mekanizmalar ve SQL Örnekleriyle Kapsamlı Teknik Rehber

İçindekiler

1. Neden PostgreSQL Bu Kadar Çok Şey Yapabiliyor?

Çoğu geliştirici PostgreSQL'i sadece bir ilişkisel veritabanı olarak tanır. Oysa doğru yapılandırıldığında MongoDB'nin yerini alan bir doküman deposu, Elasticsearch'ün yerini alan bir arama motoru, Pinecone'un yerini alan bir vektör veritabanı ve Redis/RabbitMQ'nun yerini alan bir mesaj kuyruğu olarak çalışabilir.

Bu raporun amacı, bu yeteneğin arkasındaki mimariyi en alt katmandan başlayarak SQL örnekleriyle açıklamaktır.

Temel Soru

Neden ayrı MongoDB, Redis, Elasticsearch servisleri çalıştırmak yerine tek bir PostgreSQL ile yetinilebiliyor? Cevap: PostgreSQL'in 1986'dan beri inşa edildiği genişletilebilir çekirdek mimari.

1.1 Tarihin Kısa Özeti: Stonebraker ve POSTGRES

1977'de UC Berkeley'de Michael Stonebraker liderliğinde INGRES projesi başlatıldı. Dönemin veritabanılarını yetersiz bulan Stonebraker, 1986'da POSTGRES projesini kurdu. Amacı devrimciydi:

- Veritabanı motoru, ne tür veri saklayacağını önceden bilmek zorunda olmamalı.
- Kullanıcılar, yeni veri tiplerini, yeni operatörleri ve yeni indeks yapılarını SQL ile tanımlayabilmeli.
- Motor bu yeni tipleri kendi yerleşik tipleriyle aynı şekilde işlemeli.

1996'da açık kaynak topluluk tarafından PostgreSQL adını alan bu sistem, bugün hâlâ bu felsefeyi korumaktadır. JSONB, pgvector, PostGIS — hepsi bu genişletilebilir mimarinin ürünleridir.

2. Veriler Diske Nasıl Yazılıyor? (Heap, Pages, Tuples, TOAST)

PostgreSQL'in farklı veri tiplerini aynı anda yönetebilmesini anlamak için önce fiziksel depolama katmanını kavramak gerekir.

2.1 Pages ve Heap

PostgreSQL her tabloyu disk üzerinde 8 KB'lık sabit boyutlu bloklara (page) bölerek saklar. Her page içinde birden fazla satır (tuple) bulunabilir.

```
-- Sayfa boyutunu sorgula
SHOW block_size;
-- Çıktı: 8192 (byte)

-- Bir tablonun fiziksel page sayısını gör
SELECT relpages, reltuples FROM pg_class WHERE relname = 'urunler';

-- Her satırın fiziksel konumunu gör (ctid)
SELECT ctid, * FROM urunler LIMIT 5;
-- ctid = (page_no, satir_no_icerisinde)
-- Örnek: (0,1) = 0. sayfa, 1. tuple
```

Her tuple içinde verinin yanı sıra sistem sütunları da bulunur:

Sistem Sütunu	Açıklama
ctid	Satırın fiziksel konumu (sayfa no, satır no)
xmin	Bu versiyonu yaratan transaction'ın ID'si
xmax	Bu versiyonu silen/güncelleyen transaction'ın ID'si (0 ise aktif)
tableoid	Hangi tabloya ait olduğu

2.2 TOAST: Büyük Verilerin Sırrı

8 KB'lık page sınırı büyük JSON dokümanlarını, uzun metinleri veya embedding vektörlerini saklamayı imkânsız kılıyor gibi görünebilir. İşte burada **TOAST** (The Oversized-Attribute Storage Technique) devreye girer.

TOAST üç adımda çalışır:

- Sıkıştırma dener: PostgreSQL önce PGLZ (veya LZ4) algoritmasıyla veriyi sıkıştırmayı dener. JSON gibi tekrarlayan yapılar genellikle %60-80 küçülür.
- Parçalara böler: Hâlâ sığmıyorsa, veriyi arka planda gizli bir TOAST tablosuna taşır ve ana tabloya küçük bir işaretçi (pointer) bırakır.
- Satır içi bırakır: Sıkıştırılmış hâliyle 2 KB'ın altındaysa page içinde kalır.

```
-- TOAST eşiği: 2 KB üstü veri TOAST'a gider
SHOW toast_tuple_target; -- varsayılan: 2040 byte

-- TOAST tablosunu doğrudan sorgula
SELECT relname FROM pg_class WHERE relname LIKE 'pg_toast_%';

-- Bir kolunun TOAST'ta mı, satır içinde mi olduğunu gör
SELECT attname, attstorage FROM pg_attribute
WHERE attrelid = 'makaleler'::regclass AND attnum > 0;
-- 'x' = extended (TOAST + sıkıştırma)
-- 'e' = external (TOAST, sıkıştırmasız)
-- 'm' = main (önce satır içi, zorunda kalırsa TOAST)
-- 'p' = plain (asla TOAST'a gönderilmez, örn: integer)

-- TOAST depolamayı manuel kontrol et
ALTER TABLE makaleler ALTER COLUMN icerik SET STORAGE EXTERNAL;
```

Neden Önemli?

TOAST sayesinde bir JSON sütununda 1 GB'lık bir döküman saklayabilir, o satırı sorgularken sadece istediğiniz alanı çekebilirsiniz. Büyük JSONB belgelerini MongoDB'deki gibi etkin biçimde yönetmek bu mekanizma sayesinde mümkündür.

3. MVCC: Okuyanlar Yazanları Beklemez

PostgreSQL'in hem mesaj kuyruğu hem de yoğun OLTP yüklerini aynı anda yönetebilmesinin temel sebebi Multi-Version Concurrency Control (MVCC) mimarisidir.

3.1 Klasik Kilitleme vs MVCC

Klasik veritabanlarında bir satır okunurken yazılamaz, yazılırken okunamaz — bu 'reader/writer lock' problemi ciddi performans darboğazları yaratır. MVCC bunu tersine çevirir:

- UPDATE veya DELETE yaptığınızda, eski veri yok edilmez.
- Yeni bir versiyon yaratılır. Sistemdeki her transaction kendi 'snapshot'ına bakar.
- Okuyanlar ve yazarlar birbirini hiç beklemez.

```
-- xmin ve xmax sütunlarını doğrudan sorgula
SELECT xmin, xmax, ctid, ad, fiyat FROM urunler;

-- Bir güncelleme sonrası ne olur?
UPDATE urunler SET fiyat = 999 WHERE id = 1;

-- Şimdi eski satır 'dead tuple' oldu (xmax dolu)
-- Yeni satır yeni bir ctid ile yaratıldı
-- Eski versiyonu hâlâ açık olan transaction'lar görmeye devam eder

-- Dead tuple sayısını izle
SELECT schemaname, relname, n_dead_tup, n_live_tup
FROM pg_stat_user_tables
WHERE relname = 'urunler';
```

3.2 Transaction İzolasyon Seviyeleri

İzolasyon Seviyesi	Ne Görür?
READ COMMITTED (varsayılan)	Her sorgu en son commit edilmiş veriyi görür
REPEATABLE READ	Transaction boyunca aynı snapshot'ı görür
SERIALIZABLE	Sanki işlemler sırayla yürütülmüş gibi davranır

```
-- Repeatable Read örneği
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT fiyat FROM urunler WHERE id = 1; -- 500 görür
-- Başka bir session bu arada fiyatı 999 yaptı ve commit etti
SELECT fiyat FROM urunler WHERE id = 1; -- Hâlâ 500 görür!
COMMIT;

-- Serializable ile Phantom Read koruması
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
SELECT SUM(fiyat) FROM urunler WHERE kategori = 'elektronik';
```

```
-- Başka transaction yeni ürün ekledi
SELECT SUM(fiyat) FROM urunler WHERE kategori = 'elektronik';
-- Hâlâ aynı sonuç – phantom yok
COMMIT;
```

3.3 VACUUM: Dead Tuple Temizleyici

MVCC'nin bedeli disk şişmesidir — her güncelleme yeni bir satır versiyonu yaratır, eskileri birikmeden kaldırmak gerekir. Autovacuum bu işi arka planda otomatik yapar.

```
-- Manuel vacuum
VACUUM urunler;

-- Tam vacuum (disk alanını geri kazanır, tablo kilitlenir)
VACUUM FULL urunler;

-- Vacuum ve istatistik güncelleme
VACUUM ANALYZE urunler;

-- Autovacuum ayarlarını tablo bazında özelleştir
ALTER TABLE yuksek_yazma_tablosu SET (
    autovacuum_vacuum_scale_factor = 0.01, -- %1 dead tuple'da tetikle
    autovacuum_analyze_scale_factor = 0.005 -- %0.5'te analiz et
);

-- Vacuum istatistikleri
SELECT last_vacuum, last_autovacuum, last_analyze
FROM pg_stat_user_tables WHERE relname = 'urunler';
```

4. WAL: Elektrik Kesilse Bile Veri Kaybolmaz

Write-Ahead Logging (WAL), PostgreSQL'in çökme güvencesini sağlayan mekanizmadır. Adı tam olarak anlamını anlatıyor: veriyi asıl yerine yazmadan önce bir günlüğe yaz.

4.1 WAL Nasıl Çalışır?

- Bir transaction commit edildiğinde, değişiklikler önce WAL dosyasına sıralı olarak yazılır.
- Ardından 'commit başarılı' mesajı uygulamaya döner.
- Asıl veri sayfaları (heap pages) RAM'deki buffer cache'de değiştirilmiş hâlde (dirty page) bekler.
- Checkpoint zamanı geldiğinde dirty page'ler diske kalıcı olarak yazılır.
- Sistem çökerse, son checkpoint'ten sonraki WAL kayıtları replay edilir — hiçbir veri kaybı olmaz.

```
-- WAL seviyesini görüntüle
SHOW wal_level; -- minimal, replica, logical

-- WAL dosyalarının bulunduğu dizin
SHOW data_directory; -- buraya /pg_wal ekle

-- Mevcut WAL konumunu sorgula
SELECT pg_current_wal_lsn();

-- İki WAL konumu arasındaki byte farkı
SELECT pg_wal_lsn_diff('0/3000000', '0/2000000');

-- WAL istatistikleri
SELECT * FROM pg_stat_bgwriter;
```

4.2 Checkpoint Mekanizması

Dirty page'leri sürekli yazmak yerine belirli aralıklarla toplu checkpoint yapılır. Bu hem I/O verimliliği sağlar hem de recovery süresini kısaltır.

```
-- Checkpoint aralığını ayarla
ALTER SYSTEM SET checkpoint_timeout = '10min';
ALTER SYSTEM SET max_wal_size = '2GB';

-- Manuel checkpoint tetikle
CHECKPOINT;

-- Checkpoint istatistikleri
SELECT checkpoints_timed, checkpoints_req,
       checkpoint_write_time, checkpoint_sync_time
FROM pg_stat_bgwriter;
```


4.3 Logical Replication: WAL'ı Streaming Olarak Kullan

WAL sadece crash recovery için değil, gerçek zamanlı replikasyon ve event streaming için de kullanılabilir:

```
-- Logical replication için WAL seviyesini ayarla
ALTER SYSTEM SET wal_level = 'logical';
SELECT pg_reload_conf();

-- Publication oluştur (yayıncı tarafta)
CREATE PUBLICATION my_pub FOR TABLE siparisler, urunler;

-- Subscription oluştur (abone tarafta)
CREATE SUBSCRIPTION my_sub
  CONNECTION 'host=kaynak_sunucu dbname=mydb user=repl'
  PUBLICATION my_pub;

-- Replication slot ile WAL'ı tüket (CDC için)
SELECT * FROM pg_create_logical_replication_slot('my_slot', 'pgoutput');
SELECT * FROM pg_logical_slot_get_changes('my_slot', NULL, NULL);
```

5. Index Mimarisi: Pluggable Access Methods

PostgreSQL'in en güçlü özelliklerinden biri, index yapısının değiştirilebilir (pluggable) olmasıdır. B-Tree tek seçenek değil — her problem türü için farklı bir index yapısı vardır.

5.1 B-Tree: Standart Index

Sıralı veriler, eşitlik ve aralık sorguları için. Sayılar, tarihler, metinler için varsayılan.

```
-- Standart B-Tree index
CREATE INDEX idx_urun_fiyat ON urunler (fiyat);

-- Composite index – sıralama önemli!
CREATE INDEX idx_kategori_fiyat ON urunler (kategori, fiyat DESC);

-- Partial index – sadece belirli satırlar üzerinde
CREATE INDEX idx_aktif_urunler ON urunler (fiyat)
  WHERE aktif = true;

-- Fonksiyonel index – büyük/küçük harf duyarsız arama
CREATE INDEX idx_email_lower ON kullanicilar (LOWER(email));
SELECT * FROM kullanicilar WHERE LOWER(email) = 'ali@example.com';

-- Index'i sorguda kullanmaya zorla (test için)
SET enable_seqscan = off;
EXPLAIN ANALYZE SELECT * FROM urunler WHERE fiyat = 500;
SET enable_seqscan = on;
```

5.2 GIN: Ters Çevrilmiş Index (JSONB ve Full-Text Search için)

GIN (Generalized Inverted Index), bir değer içindeki tüm elemanları listeleterek ters çevrilmiş bir harita oluşturur. Kitap sonu dizini gibi düşün: hangi kelime hangi sayfalarda geçiyor.

```
-- JSONB için GIN index
CREATE INDEX idx_veri_gin ON urunler USING GIN (veri);

-- İçeriyor mu? (@>) operatörü – GIN'i kullanır
SELECT * FROM urunler WHERE veri @> '{"renk": "mavi"}';

-- Anahtar varlığını kontrol et (?)
SELECT * FROM urunler WHERE veri ? 'stok';

-- Herhangi biri mevcut mu? (?!|)
SELECT * FROM urunler WHERE veri ?| ARRAY['stok', 'depo'];

-- Full-text search için GIN
CREATE INDEX idx_icerik_gin ON makaleler USING GIN (to_tsvector('turkish',
icerik));

-- Array sütunlar için GIN
```

```
CREATE INDEX idx_etiketler_gin ON makaleler USING GIN (etiketler);
SELECT * FROM makaleler WHERE etiketler @> ARRAY['postgresql', 'veritabani'];
```

5.3 GiST: Geometri ve Aralıklar için

GiST (Generalized Search Tree), tam eşleşme değil 'yakınlık' ve 'örtüşme' sorgularını destekleyen esnek bir index yapısıdır. PostGIS, tam metin araması ve aralık tiplerinde kullanılır.

```
-- Aralık tipi ve GiST index
CREATE TABLE rezervasyonlar (
  id SERIAL PRIMARY KEY,
  oda_id INT,
  donem TSRANGE
);
CREATE INDEX idx_donem_gist ON rezervasyonlar USING GiST (donem);

-- Çakışan rezervasyonları bul (&&)
SELECT * FROM rezervasyonlar
WHERE donem && '[2024-06-01, 2024-06-07)>::tsrange;

-- Tsvector için GiST (GIN'den yavaş ama daha az yer kaplar)
CREATE INDEX idx_fts_gist ON makaleler USING GiST (arama_vektoru);

-- pg_trgm ile fuzzy search
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_ad_trgm ON urunler USING GiST (ad gin_trgm_ops);
SELECT * FROM urunler WHERE ad % 'Laptob'; -- 'Laptop'u bulur'
```

5.4 BRIN: Büyük Tablolar için Küçük Index

BRIN (Block Range INdex), her sayfa bloğundaki değerlerin min/max'ını saklar. Çok küçük bir index dosyasıyla sıralı yazılan zaman serisi verilerinde etkilidir.

```
-- Zaman serisi tablosu için BRIN
CREATE TABLE olaylar (
  id BIGSERIAL PRIMARY KEY,
  zaman TIMESTAMPTZ DEFAULT NOW(),
  veri JSONB
);
CREATE INDEX idx_zaman_brin ON olaylar USING BRIN (zaman);

-- B-Tree ile karşılaştırma:
-- B-Tree index: tablonun %5-10'u kadar yer
-- BRIN index: birkaç KB – tablonun %0.001'i bile değil

-- BRIN pages_per_range ayarı
CREATE INDEX idx_zaman_brin ON olaylar USING BRIN (zaman)
WITH (pages_per_range = 32);
```

5.5 HNSW: Vektör Araması için

HNSW (Hierarchical Navigable Small World), pgvector extension'ı ile gelen graf tabanlı bir index yapısıdır. Yüksek boyutlu vektörler arasında yaklaşık en yakın komşu aramayı milisaniyelere indirir.

```
CREATE EXTENSION vector;

-- HNSW index oluştur
CREATE INDEX idx_embedding_hnsw ON makaleler
  USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
-- m: her node'un max bağlantısı (yüksek = doğru ama yavaş build)
-- ef_construction: arama kalitesi (yüksek = iyi ama build yavaşlar)

-- IVFFlat: Alternatif – daha az bellek, biraz daha az doğru
CREATE INDEX idx_embedding_ivf ON makaleler
  USING ivfflat (embedding vector_l2_ops)
  WITH (lists = 100);

-- Arama kalitesini ayarla (runtime)
SET hnsw.ef_search = 100; -- Daha yüksek = daha doğru ama yavaş

-- Üç farklı mesafe metriği:
-- <=> Cosine distance (normalize edilmiş vektörler için)
-- <-> Euclidean / L2 distance
-- <#> Inner product (dot product, negatif)
```

6. Extension Sistemi: Çekirdeği Değiştirmeden Her Şeyi Değiştir

PostgreSQL'in en büyük mimarî kararı: yeni veri tiplerini, operatörleri ve index yapılarını C veya SQL ile yazılmış shared library olarak yükleyebilmek.

6.1 Extension Nasıl Çalışır?

```
-- Extension yükleme
CREATE EXTENSION pgvector;
CREATE EXTENSION pg_trgm;
CREATE EXTENSION hstore;
CREATE EXTENSION PostGIS;
CREATE EXTENSION pg_stat_statements;

-- Yüklü extension'ları listele
SELECT name, default_version, installed_version
FROM pg_available_extensions
WHERE installed_version IS NOT NULL;

-- Extension'ın ne eklediğini gör
SELECT * FROM pg_extension WHERE extname = 'vector';

-- Extension kaldır
DROP EXTENSION pg_trgm;
```

6.2 Kendi Veri Tipini Yaz

Extension sistemi sayesinde PostgreSQL'e tamamen yeni bir veri tipi öğretebilirsin — ve bu tip, int veya text gibi native tiplerle aynı şekilde davranır:

```
-- Basit bir özel tip tanımla
CREATE TYPE para AS (
    miktar NUMERIC(15,2),
    para_birimi CHAR(3)
);

-- Bu tipi kullanan bir tablo
CREATE TABLE faturalar (
    id SERIAL PRIMARY KEY,
    tutar para
);

-- Composite type olarak kullan
INSERT INTO faturalar (tutar) VALUES (ROW(1500.00, 'TRY'));
SELECT (tutar).miktar, (tutar).para_birimi FROM faturalar;

-- Domain: kısıtlı tip
CREATE DOMAIN pozitif_sayi AS NUMERIC
CHECK (VALUE > 0);
```

```
CREATE TABLE stok (  
    urun_id INT,  
    adet pozitif_sayi -- Otomatik kısıtlama  
);
```

7. Document Store: JSONB ile MongoDB Alternatifi

PostgreSQL'in JSONB tipi, JSON verilerini binary formatta saklar. Bu sayede hem tam SQL sorguları hem de belge tabanlı sorgular aynı veri üzerinde çalışabilir.

7.1 JSON vs JSONB: Farkları

Özellik	JSON vs JSONB
Saklama	JSON: düz metin JSONB: ayrıştırılmış binary
Yazma hızı	JSON: daha hızlı JSONB: biraz yavaş (ayrıştırma)
Okuma hızı	JSON: yavaş JSONB: çok hızlı
Index desteği	JSON: yok JSONB: GIN index
Sütun sırası	JSON: korunur JSONB: korunmaz
Boşluklar	JSON: korunur JSONB: kaldırılır
Önerilen	Neredeyse her zaman JSONB kullan

7.2 Temel JSONB Operatörleri

```
CREATE TABLE urunler (
  id SERIAL PRIMARY KEY,
  veri JSONB
);

INSERT INTO urunler (veri) VALUES
  ('{"isim": "Laptop", "fiyat": 1500, "stok": {"A": 10, "B": 5}, "renkler": ["siyah", "gri"]}'),
  ('{"isim": "Telefon", "fiyat": 800, "renkler": ["beyaz", "mavi"]}');

-- Alan erişimi
SELECT veri->'isim'      FROM urunler; -- JSON döner: "Laptop"
SELECT veri->>'isim'     FROM urunler; -- Text döner: Laptop
SELECT veri->'stok'->'A' FROM urunler; -- Nested: 10

-- İçeriyor mu? (@>)
SELECT * FROM urunler WHERE veri @> '{"fiyat": 1500}';

-- Anahtar var mı? (?)
SELECT * FROM urunler WHERE veri ? 'stok';

-- JSON path ile sorgula
SELECT * FROM urunler
WHERE veri @? '$.renkler[*] ? (@ == "siyah)';

-- jsonpath ile değer çıkar
SELECT jsonb_path_query(veri, '$.renkler[*]') FROM urunler;
```

7.3 JSONB Güncelleme

```
-- Alan ekle / güncelle (||)
UPDATE urunler
SET veri = veri || '{"aktif": true}'
WHERE id = 1;

-- Alan sil (-)
UPDATE urunler
SET veri = veri - 'aktif'
WHERE id = 1;

-- Nested güncelleme (jsonb_set)
UPDATE urunler
SET veri = jsonb_set(veri, '{stok, A}', '20')
WHERE id = 1;

-- Array'e eleman ekle (jsonb_insert)
UPDATE urunler
SET veri = jsonb_insert(veri, '{renkler, -1}', '"kirmizi"')
WHERE id = 1;

-- Atomic upsert – JSON belgesi yoksa ekle, varsa güncelle
INSERT INTO urunler (id, veri)
VALUES (1, '{"isim": "Laptop", "fiyat": 2000}')
ON CONFLICT (id)
DO UPDATE SET veri = urunler.veri || EXCLUDED.veri;
```

7.4 JSONB Aggregation

```
-- JSON nesne oluştur
SELECT jsonb_build_object(
    'toplam_urun', COUNT(*),
    'ortalama_fiyat', AVG(veri->>'fiyat')::numeric,
    'max_fiyat', MAX(veri->>'fiyat')::numeric
) FROM urunler;

-- JSON array oluştur
SELECT jsonb_agg(veri ORDER BY (veri->>'fiyat')::int DESC)
FROM urunler;

-- JSONB'yi ilişkisel tabloya dönüştür
SELECT key, value
FROM urunler,
     jsonb_each(veri)
WHERE id = 1;
```


8. Search Engine: Full-Text Search ile Elasticsearch Alternatifi

PostgreSQL, metinleri kelime köklerine ayıran (stemming), eş anlamlıları bulan ve alaka puanı hesaplayan tam metin araması motoruna sahiptir.

8.1 tsvector ve tsquery

```
-- tsvector: metnin aranabilir temsili
SELECT to_tsvector('turkish', 'PostgreSQL güçlü bir veritabanı yönetim
sistemidir');
-- 'güçlü':2 'postgresql':1 'sistemi':5 'veritabanı':4 'yönetim':5

-- tsquery: arama sorgusu
SELECT to_tsquery('turkish', 'veritabanı & yönetim');

-- Eşleşme kontrolü (@@)
SELECT to_tsvector('turkish', 'veritabanı yönetim sistemi')
@@ to_tsquery('turkish', 'veritabanı');
-- true

-- plainto_tsquery: doğal dil sorgusu
SELECT plainto_tsquery('turkish', 'postgresql veritabanı');
-- 'postgresql' & 'veritabanı'

-- phraseto_tsquery: kelime sırasına duyarlı
SELECT phraseto_tsquery('turkish', 'veri tabanı yönetimi');
```

8.2 Tam Metin Araması Kurulumu

```
CREATE TABLE makaleler (
  id SERIAL PRIMARY KEY,
  baslik TEXT,
  icerik TEXT,
  arama_vektoru TSVECTOR -- computed sütun
);

-- Otomatik güncelleme için trigger
CREATE OR REPLACE FUNCTION makaleler_arama_guncelle()
RETURNS TRIGGER AS $$
BEGIN
  NEW.arama_vektoru :=
    setweight(to_tsvector('turkish', COALESCE(NEW.baslik, '')), 'A') ||
    setweight(to_tsvector('turkish', COALESCE(NEW.icerik, '')), 'B');
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_arama
BEFORE INSERT OR UPDATE ON makaleler
FOR EACH ROW EXECUTE FUNCTION makaleler_arama_guncelle();
```

```
-- GIN index
CREATE INDEX idx_arama ON makaleler USING GIN (arama_vektoru);
```

8.3 Sıralama ve Vurgulama

```
-- Alaka puanıyla sırala
SELECT baslik,
       ts_rank(arama_vektoru, sorgu) AS puan,
       ts_rank_cd(arama_vektoru, sorgu) AS puan_cd
FROM makaleler,
     to_tsquery('turkish', 'postgresql & performans') sorgu
WHERE arama_vektoru @@ sorgu
ORDER BY puan DESC;

-- Arama terimini metinde vurgula
SELECT ts_headline(
       'turkish',
       icerik,
       to_tsquery('turkish', 'postgresql'),
       'MaxWords=20, MinWords=10, StartSel=<b>, StopSel=</b>'
) AS vurgulu
FROM makaleler
WHERE arama_vektoru @@ to_tsquery('turkish', 'postgresql');

-- Fuzzy search: pg_trgm ile yazım hatalarına toleranslı
CREATE EXTENSION pg_trgm;
SELECT baslik, similarity(baslik, 'Postgreesql') AS benzerlik
FROM makaleler
WHERE baslik % 'Postgreesql' -- benzerlik > 0.3
ORDER BY benzerlik DESC;
```

9. Vector Database: pgvector ile Semantik Arama

Yapay zeka uygulamalarında (RAG, öneri sistemleri, görsel arama) metinlerin veya nesnelerin vektör temsillerini (embedding) saklamak ve benzerlik araması yapmak gerekir. pgvector bunu doğrudan PostgreSQL içinde sağlar.

9.1 Kurulum ve Temel Kullanım

```
CREATE EXTENSION vector;

-- Embedding sütunlu tablo
CREATE TABLE makaleler (
  id SERIAL PRIMARY KEY,
  baslik TEXT,
  icerik TEXT,
  embedding VECTOR(1536) -- OpenAI text-embedding-3-small boyutu
);

-- Vektör ekle (Python'da OpenAI API'den alınan embedding)
INSERT INTO makaleler (baslik, icerik, embedding)
VALUES ('PostgreSQL mimarisi', '...', '[0.1, 0.3, ...]');

-- En yakın 5 makaleyi bul (cosine distance)
SELECT baslik, 1 - (embedding <=> '[0.2, 0.4, ...]':vector) AS benzerlik
FROM makaleler
ORDER BY embedding <=> '[0.2, 0.4, ...]':vector
LIMIT 5;
```

9.2 Mesafe Metrikleri

Operatör	Mesafe Tipi ve Kullanım Alanı
<=>	Cosine distance — yön benzerliği, metin embeddinglerde tercih edilir
<->	Euclidean (L2) distance — geometrik uzaklık, görsel/ses embeddinglerde
<#>	Negative inner product — dot product, normalize vektörlerde cosine ile eşdeğer
<+>	L1 (Manhattan) distance — pgvector 0.7+ ile

9.3 Hybrid Search: Full-Text + Vektör Aynı Anda

```
-- Full-text ve semantik aramayı birleştir (RRF skoru)
WITH fts AS (
  SELECT id, ts_rank(arama_vektoru, sorgu) AS skor
  FROM makaleler,
  to_tsquery('turkish', 'postgresql & mimari') sorgu
  WHERE arama_vektoru @@ sorgu
```

```
),
semantic AS (
  SELECT id,
         1 - (embedding <=> '[0.2, 0.4, ...]':vector) AS skor
  FROM makaleler
  ORDER BY embedding <=> '[0.2, 0.4, ...]':vector
  LIMIT 20
)
SELECT m.baslik,
       COALESCE(fts.skor, 0) * 0.3 +
       COALESCE(sem.skor, 0) * 0.7 AS toplam_skor
FROM makaleler m
LEFT JOIN fts ON m.id = fts.id
LEFT JOIN semantic sem ON m.id = sem.id
ORDER BY toplam_skor DESC
LIMIT 10;
```

9.4 Metadata Filtrelemeli Vektör Araması

```
-- Hem benzerlik hem de filtre
SELECT id, baslik, kategori,
       embedding <=> '[0.1, 0.5, ...]':vector AS distance
FROM makaleler
WHERE kategori = 'teknoloji'
  AND created_at > NOW() - INTERVAL '30 days'
ORDER BY embedding <=> '[0.1, 0.5, ...]':vector
LIMIT 10;

-- JSONB metadata ile filtre
SELECT id, baslik,
       embedding <=> query_vec AS distance
FROM makaleler,
     (SELECT '[0.1, 0.5, ...]':vector AS query_vec) q
WHERE meta @> '{"dil": "tr", "onaylandi": true}'
ORDER BY distance
LIMIT 10;
```

10. Message Queue: LISTEN/NOTIFY ve SKIP LOCKED

PostgreSQL, hem gerçek zamanlı pub/sub mesajlaşması hem de dayanıklı iş kuyruğu (job queue) için kullanılabilir. İki farklı mekanizma farklı ihtiyaçları karşılar.

10.1 LISTEN / NOTIFY: Gerçek Zamanlı Pub/Sub

Aynı veritabanına bağlı farklı sessionlar arasında anlık mesaj göndermek için kullanılır. Mesajlar kalıcı değildir — alıcı bağlı değilse mesaj kaybolur.

```
-- Abone ol (consumer session)
LISTEN siparis_kanali;
LISTEN stok_guncelleme;

-- Mesaj yayınla (producer session)
NOTIFY siparis_kanali, '{"siparis_id": 42, "musteri": "Ali"}';

-- Transaction içinde notify (commit'te tetiklenir)
BEGIN;
INSERT INTO siparisler (musteri_id, toplam) VALUES (1, 250);
NOTIFY siparis_kanali, '{"yeni_siparis": true}';
COMMIT;

-- pg_notify fonksiyonu (trigger içinde kullanışlı)
SELECT pg_notify('stok_guncelleme', json_build_object(
    'urun_id', 15, 'adet', 5)::text
);

-- Aktif listen sessionları izle
SELECT pid, application_name, query, state
FROM pg_stat_activity
WHERE query LIKE '%LISTEN%';
```

Python'da LISTEN Kullanımı

```
import psycopg2; conn.autocommit = True; cur.execute('LISTEN kanal'); import select;
select.select([conn],[],[]) — bu kadar. Mesaj geldiğinde conn.notifies listesi dolar.
```

10.2 SKIP LOCKED: Dayanıklı Job Queue

Mesajların kaybolmaması gerekiyorsa (task queue, email kuyruğu, ödeme işlemleri) LISTEN/NOTIFY yetmez. Bunun için SKIP LOCKED ile tabloya dayalı bir kuyruk kullanılır.

```
-- Kuyruk tablosu
CREATE TABLE is_kuyrugu (
    id BIGSERIAL PRIMARY KEY,
    is_tipi TEXT NOT NULL,
    payload JSONB NOT NULL,
    durum TEXT DEFAULT 'bekliyor'
```

```
    CHECK (durum IN ('bekliyor', 'isleniyor', 'tamamlandi', 'hata')),
    hata_mesaji TEXT,
    deneme_sayisi INT DEFAULT 0,
    olusturulma TIMESTAMPTZ DEFAULT NOW(),
    isleme_baslama TIMESTAMPTZ,
    isleme_bitis TIMESTAMPTZ
);

-- Index: bekleyen işleri hızlıca bul
CREATE INDEX idx_kuyruk_durum ON is_kuyrugu (durum, olusturulma)
    WHERE durum = 'bekliyor';

-- İş ekle
INSERT INTO is_kuyrugu (is_tipi, payload)
VALUES ('email_gonder', '{"to": "ali@example.com", "konu": "Hosgeldiniz"}');

-- Worker: iş al (SKIP LOCKED kritik!)
BEGIN;
SELECT * FROM is_kuyrugu
WHERE durum = 'bekliyor'
ORDER BY olusturulma
LIMIT 1
FOR UPDATE SKIP LOCKED; -- Kilitli satırları atla, beklemeden geç

-- İş başlatıldı olarak işaretle
UPDATE is_kuyrugu
SET durum = 'isleniyor',
    isleme_baslama = NOW()
WHERE id = <alınan_id>;
COMMIT;

-- İş tamamlandı
UPDATE is_kuyrugu
SET durum = 'tamamlandi', isleme_bitis = NOW()
WHERE id = <id>;

-- İş hata verdi – retry için
UPDATE is_kuyrugu
SET durum = 'bekliyor',
    deneme_sayisi = deneme_sayisi + 1,
    hata_mesaji = 'Bağlantı hatası'
WHERE id = <id> AND deneme_sayisi < 3;
```

10.3 SKIP LOCKED ile Partitioned Queue

```
-- Öncelikli kuyruk: önce yüksek öncelikli işler
SELECT * FROM is_kuyrugu
WHERE durum = 'bekliyor'
ORDER BY oncelik DESC, olusturulma ASC
LIMIT 1
FOR UPDATE SKIP LOCKED;

-- Tip bazında paralel worker'lar
-- Worker 1: sadece email işlerini al
SELECT * FROM is_kuyrugu
```

```
WHERE durum = 'bekliyor' AND is_tipi = 'email_gonder'
LIMIT 5 FOR UPDATE SKIP LOCKED;

-- Worker 2: sadece rapor işlerini al
SELECT * FROM is_kuyruğu
WHERE durum = 'bekliyor' AND is_tipi = 'rapor_uret'
LIMIT 2 FOR UPDATE SKIP LOCKED;

-- Takılı kalan işleri (stale) temizle
UPDATE is_kuyruğu
SET durum = 'bekliyor', isleme_baslama = NULL
WHERE durum = 'isleniyor'
    AND isleme_baslama < NOW() - INTERVAL '30 minutes';
```

11. Gelişmiş SQL: Window Fonksiyonları, CTE ve Upsert

11.1 Window Fonksiyonları

Uygulama tarafında döngüyle yapılan sıralamalar, toplam hesapları ve karşılaştırmalar veritabanı içinde çok daha verimli yapılabilir.

```
-- Kategori içinde sıralama
SELECT ad, kategori, fiyat,
       RANK() OVER (PARTITION BY kategori ORDER BY fiyat DESC) AS siralama,
       DENSE_RANK() OVER (PARTITION BY kategori ORDER BY fiyat DESC) AS
yogun_siralama
FROM urunler;

-- Çalışan toplam (running total)
SELECT siparis_tarihi, toplam,
       SUM(toplam) OVER (ORDER BY siparis_tarihi) AS kumulatif_toplam
FROM siparisler;

-- Lag/Lead: bir önceki/sonraki satırla karşılaştır
SELECT tarih, satis,
       LAG(satis) OVER (ORDER BY tarih) AS onceki_gun,
       satis - LAG(satis) OVER (ORDER BY tarih) AS degisim
FROM gunluk_satis;

-- Moving average (hareketli ortalama)
SELECT tarih, deger,
       AVG(deger) OVER (
         ORDER BY tarih
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS haftalik_ort
FROM metrikler;

-- NTILE: gruplara böl
SELECT ad, fiyat,
       NTILE(4) OVER (ORDER BY fiyat) AS ceyrekte
FROM urunler;
```

11.2 CTE (Common Table Expressions)

```
-- Basit CTE – okunabilirlik için
WITH yuksek_degerli_musteriler AS (
  SELECT muster_i_id, SUM(toplam) AS toplam_harcama
  FROM siparisler
  GROUP BY muster_i_id
  HAVING SUM(toplam) > 10000
),
musteri_detaylari AS (
  SELECT m.*, ydm.toplam_harcama
  FROM musteriler m
  JOIN yuksek_degerli_musteriler ydm ON m.id = ydm.muster_i_id
```



```
)
SELECT * FROM musteri_detaylari ORDER BY toplam_harcama DESC;

-- Recursive CTE – hiyerarşik yapılar
WITH RECURSIVE kategori_agaci AS (
  -- Base case: kök kategoriler
  SELECT id, ad, ust_id, 0 AS seviye, ad::TEXT AS yol
  FROM kategoriler WHERE ust_id IS NULL

  UNION ALL

  -- Recursive case: alt kategoriler
  SELECT k.id, k.ad, k.ust_id, ka.seviye + 1,
         ka.yol || ' > ' || k.ad
  FROM kategoriler k
  JOIN kategori_agaci ka ON k.ust_id = ka.id
)
SELECT seviye, yol FROM kategori_agaci ORDER BY yol;
```

11.3 Upsert ve RETURNING

```
-- Upsert: yoksa ekle, varsa güncelle
INSERT INTO urunler (id, ad, fiyat, stok)
VALUES (1, 'Laptop Pro', 2000, 50)
ON CONFLICT (id)
DO UPDATE SET
  fiyat = EXCLUDED.fiyat,
  stok = EXCLUDED.stok,
  guncelleme = NOW();

-- RETURNING: eklenen/güncellenen veriyi geri al
INSERT INTO siparisler (musteri_id, toplam)
VALUES (1, 500)
RETURNING id, olusturulma;

-- Güncelleme sonrası hem eski hem yeni değeri al
WITH guncelleme AS (
  UPDATE urunler SET stok = stok - 1
  WHERE id = 1 AND stok > 0
  RETURNING id, stok AS yeni_stok
)
SELECT * FROM guncelleme;

-- ON CONFLICT DO NOTHING: çakışmayı yoksay
INSERT INTO kullanıcı_etiket (kullanıcı_id, etiket)
VALUES (1, 'admin')
ON CONFLICT (kullanıcı_id, etiket) DO NOTHING;
```

12. Performans Analizi: EXPLAIN ANALYZE ve Query Planner

12.1 EXPLAIN ANALYZE

```
-- Temel explain
EXPLAIN SELECT * FROM urunler WHERE fiyat > 500;

-- Gerçek execution ile (veriyi çalıştırır)
EXPLAIN ANALYZE SELECT * FROM urunler WHERE fiyat > 500;

-- Detaylı output
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT u.ad, COUNT(s.id) AS siparis_sayisi
FROM urunler u
LEFT JOIN siparis_satirlari s ON u.id = s.urun_id
GROUP BY u.id, u.ad
ORDER BY siparis_sayisi DESC;

-- JSON formatında (pgAdmin/IDE'ler için)
EXPLAIN (ANALYZE, FORMAT JSON)
SELECT * FROM makaleler WHERE arama_vektoru @@ to_tsquery('turkish',
'postgresql');
```

12.2 pg_stat_statements: Yavaş Sorguları Bul

```
CREATE EXTENSION pg_stat_statements;

-- En yavaş 10 sorgu
SELECT query,
       calls,
       total_exec_time / calls AS ort_sure_ms,
       rows / calls AS ort_satir
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;

-- Index kullanılmayan sorgular
SELECT schemaname, relname, seq_scan, seq_tup_read,
       idx_scan, idx_tup_fetch
FROM pg_stat_user_tables
WHERE seq_scan > idx_scan
ORDER BY seq_scan DESC;

-- Eksik index önerisi
SELECT schemaname, relname, attname,
       n_distinct, correlation
FROM pg_stats
WHERE tablename = 'urunler';
```

12.3 Connection Pooling: PgBouncer

Serverless mimariler ve yüksek trafikli uygulamalarda her request yeni bir veritabanı bağlantısı açmak PostgreSQL'i çökertebilir. PgBouncer bağlantıları havuzlar.

```
-- Mevcut bağlantıları izle
SELECT count(*), state FROM pg_stat_activity GROUP BY state;

-- Max connection sayısını gör
SHOW max_connections; -- varsayılan: 100

-- Bağlantı başına bellek kullanımı ~5-10 MB
-- 1000 bağlantı = ~5-10 GB bellek (!) = PgBouncer gerekli

-- PgBouncer modları:
-- session: her client bir connection alır (uyumlu ama verimsiz)
-- transaction: transaction bitince connection havuza döner (önerilen)
-- statement: her statement'ta döner (prepared statement'larla uyumsuz)
```

13. PostgreSQL Ne Zaman Yetmez?

Dürüst olmak gerekirse: PostgreSQL her şeyin yerini alamaz. Şu durumlarda özel araçlara geçmek daha mantıklıdır:

İhtiyaç	Neden PostgreSQL Yetmez / Alternatif
Saf önbellek (Cache)	Redis: microsaniye latency, veri tipi zenginliği (sorted set, hyperloglog). PostgreSQL'den 10-100x hızlı.
Ekstrem yazma yükü (Sharding)	CockroachDB, YugabyteDB: yatay ölçekleme. PG tek node'la sınırlı.
Ağır analitik (OLAP)	ClickHouse, DuckDB: sütun bazlı depolama. Milyar satır aggregation'da 100x hızlı.
Mesaj streaming	Kafka: milyonlarca msg/sn, çok consumer, uzun süreli saklama. PG queue bunu taşımaz.
Grafik veritabanı	Neo4j, AGE (PG extension): derin graph traversal. PG'nin recursive CTE'si sınırlı kalır.
Time-series (yoğun)	TimescaleDB (PG extension olarak da var) veya InfluxDB: otomatik partitioning, compression.

Pratik Öneri

Projeye PostgreSQL ile başlayın. Gerçek ölçek problemleriyle karşılaşıldığında, o spesifik ihtiyaç için özel araç ekleyin. 'Belki lazım olur' diye baştan Redis + Kafka + Elasticsearch kurmak bakım yükünü 3x artırır.

14. Özet: Her Şey Bir Arada Nasıl Çalışıyor?

PostgreSQL'in bu kadar çok şeyi yapabilmesinin ardında birbirini tamamlayan beş mimarî karar yatar:

Mimari Karar	Ne Sağladığı
Genişletilebilir tip sistemi	JSONB, VECTOR, GEOMETRY gibi tipler çekirdek değiştirmeden eklenir
Pluggable index yapısı	B-Tree, GIN, GiST, HNSW — her veri tipi için optimal index
MVCC	Okuyanlar ve yazarlar birbirini beklemez, SKIP LOCKED ile queue mümkün
WAL	Elektrik kesilse veri kaybolmaz, logical replication ile streaming
TOAST	GB'lık JSON ve embedding'ler sayfa sınırını aşmadan saklanır

Bu beş mekanizma birlikte çalıştığında ortaya şu çıkıyor: tek bir SQL bağlantısıyla hem doküman sorgulayabilir, hem embedding araması yapabilir, hem full-text search kullanabilir, hem kuyruktan iş çekebilirsiniz — hepsi ACID garantisiyle, tek bir transaction içinde.

— Rapor Sonu —